Jakob Schouren

Betriebssysteme, Übungsblatt 3, Wintersemester 2025/26

ZFS ist ein transaktionales 128-bit-Dateisystem, welches die Adressierung von 10^{21} Bytes erlaubt und sich insbesondere durch die Kombination des Copy-On-Write-Verfahrens, eines Logical Volume Managers und Software-RAIDs von seinen Vorgängern unterscheidet. CoW bedeutet in diesem Zusammenhang, dass beim Schreiben einer Datei die alten Datenblöcke niemals direkt überschrieben werden, sondern zunächst eine Kopie an einer freien Stelle erstellt wird. Nach erfolgreichem Schreibvorgang der neuen Blöcke werden dann lediglich die Verweise in den Metadaten verändert. Dadurch wird nicht nur die Dateikonsistenz bei Systemausfällen gewährleistet, sondern auch das Erstellen von Snapshots erleichtert. Neben dessen verfolgt ZFS als Entwurfsziele die Zusammenfassung von physikalischen Speichermedien in Pools, die uneingeschränkte Gewährleistung von Datenintegrität bzw. das Verhindern von Datenverlusten sowie die in diesem Rahmen die höchstmögliche Geschwindigkeit bereitzustellen. Letzteres wird unter anderem durch ein fortschrittliches Caching-System bestehend aus ARC, L2ARC und ZIL erreicht. Datenintegrität wird insbesondere dadurch gewährleistet, dass jedem allokierten Block durch einen wählbaren Algorithmus eine Prüfsumme zugewiesen wird. Beim Lesen der Blöcke wird dadurch jegliche Datenkorruption erkannt und bei entsprechender Redundanz ggf. sogar automatisch repariert. ZFS abstrahiert physikalische Festplatten in Form von vdevs und Pools. Während ein vdev im einfachsten Fall eine Platte und im Fall von raid(z) eine Gruppe von Platten darstellt, fasst ein Pool ein oder mehrere vdevs zusammen. Innerhalb des Pools können anschließend Datasets und Volumes erstellt und mit Daten angereichert werden, wobei sie sich den verfügbaren Speicherplatz des Pools teilen müssen.[1]

Aufgabe 1: Datensicherung und -archivierung mit der Hilfe von ZFS

Das implementierte Datensicherungsprogramm ist in python verfasst und sichert bei seiner Ausführung alle rekursiven Inhalte eines Datasets als Snapshot an einem anderen Ort bzw. in einem anderen Dataset. Anschließend werden basierend auf einem Retention-Count alle überzähligen Snapshots aus dem Backup-Dataset gelöscht sowie alle bis auf den neuesten Snapshot aus dem zu sichernden Dataset. Die verbleibende Kopie ermöglicht einen inkrementellen Ansatz bei der Erstellung der Snapshots.
Wichtig: Das Skript muss mit sudo-Berechtigungen bzw. als root-User ausgeführt werden. Wird eine regelmäßige Sicherung gewünscht, kann dies über Cronjobs bzw.

[1] ZFS FreeBSD (https://docs.freebsd.org/de/books/handbook/zfs/, Zugriff vom 20.03.2026), ZFS on Linux ubuntuusers (https://wiki.ubuntuusers.de/Archiv/ZFS_on_Linux/, Zugriff vom 20.03.2026), Checksums OpenZFS (https://openzfs.github.io/openzfs-docs/Basic%20Concepts/Checksums.html, Zugriff vom 20.03.2026), System Administration OpenZFS (https://openzfs.org/wiki/System_Administration, Zugriff vom 20.03.2026)

einen Eintrag ins Crontab von root erreicht werden. Folgender Befehl erstellt alle 5 Minuten einen Snapshot und löscht alle über 10 hinausgehenden Kopien:

**/5 * * * * python3 /path/to/folder/backup.py 10 >> /var/log/zfs-snapshot.log 2>&1*

Die Anzahl der verbleibenden Snapshots, den zu sichernden Ordner sowie den Backup-Ordner lassen sich beim Aufruf des Skripts als Kommandozeilenparameter in der Form *python3 backup.py 10 pool/dataset pool/dataset* übergeben. Wird kein Parameter übergeben, werden Default-Werte angewendet, im Falle des Skripts, ein Retention-Count von 5 sowie die Datasets *zpool_for-saving/save-me* und *zpool_for-backups/backups*. Die verwendeten Pools und Datasets müssen zum Zeitpunkt der Skriptausführung bereits vorhanden sein, ansonsten weist das Skript darauf hin, diese vor der Ausführung zu erstellen. Um einen zpool erstellen zu können, benötigt man mindestens ein freies Block Device. Diese lassen sich mit *lsblk* auflisten. Als Block Devices kommen Files, Partitionen als auch ganze Laufwerke in Frage[2]. Mit dem Befehl *sudo truncate -s 5GB /home/ubuntu/disk.img* lässt sich bspw. eine Image-Datei erstellen, die dann als Loop-Device verwendet werden kann. Dafür lässt man sich mit *losetup -f* das nächste freie Loop-Device ausgeben und weist die „Disk“ diesem Device z.B. mit *losetup /dev/loop17 /home/ubuntu/disk.img* zu. Vorteil dieser Herangehensweise ist, dass man vergleichsweise einfach und schnell die Funktionalitäten von ZFS testen kann. Allerdings verfallen die Loop-Devices nach einem Neustart des Systems ohne weitergehende Konfigurationen, wodurch man das Einbinden als Block-Device wiederholen muss. Alternativ kann man aber auch eine ganze Platte verwenden. In der Virtualbox-Software lassen sich Platten unter Storage für jede VM einfach neu erstellen und einbinden. Nach einem Neustart des Systems, sind diese dann dauerhaft unter /dev/sd[a-z] zugänglich und können anschließend als zpool genutzt werden. ZFS verwendet für die Administration die beiden Befehle zpool und zfs. Über *zpool create mypool /dev/sda* lässt sich z.B. ein neuer Pool namens „mypool“ erstellen, welcher aus einem einzelnen vdev bzw. einem einzigen Block Device besteht, also keine Redundanz aufweist. Gibt man dabei keinen expliziten Mountpoint an, wird für den Pool automatisch ein gleichnamiges Dataset angelegt. Über zfs create lassen sich anschließend weitere Datasets bzw. Filesysteme erstellen. Da im Rahmen des Programms keine expliziten Mountpoint für die erstellten Datasets angegeben wurden, sind die Datasets im Root-Verzeichnis gemountet.

Um innerhalb des Programms Kommandozeilenbefehle über zfs abzusetzen, wird eine Funktion *run_shell_command(command)* eingeführt, welche als Input einen Befehl als String annimmt, diesen anschließend über *subprocess.run()* ausführt und das entsprechende Ergebnis zurückgibt. Um einen einzigartigen Snapshot-Namen zu erzeugen, werden in der Funktion *create_snapshot(dataset, timestamp)* der betroffene Pool, das betroffene Dataset sowie der aktuelle Timestamp im Format *pool/dataset@timestamp* zusammengefügt. Anschließend wird über *zfs snapshot*

[2] ZPool Ubuntu (https://wiki.ubuntu.com/ZFS/ZPool, Zugriff vom 21.03.2026)

pool/dataset@timestamp der Snapshot erstellt. In *transfer_snapshot(snap, pred, dataset)* wird der Snapshot schließlich über die Funktionen send und *recv/receive* auf das Backup-Dataset übertragen. Sind bereits Snapshots vorhanden, wird der direkte Vorgänger des aktuellen Snapshots mit dem Parameter -i übergeben, was zu einer erhöhten Speichereffizienz führt, da nur die Differenz der Blöcke in beiden Snapshots übertragen wird. Um nach abgeschlossener Übertragung alle überschüssigen Snapshots zu löschen, wird in *get_all_snapshots(dataset)* der Befehl *zfs list -H -t snapshot -s creation -o name dataset* ausgeführt, wodurch für ein bestimmtes Dataset alle Namen der ZFS-Objekte des Typs Snapshot ohne Header und sortiert nach dem Creation-Date zeilenweise ausgegeben werden. Über *strip()* und *splitlines()* lassen sich die Snapshots anschließend als bereits sortierte Liste speichern. In *delete_snapshot_overflow(dataset, retention)* werden anschließend alle über den Retention-Count hinausgehenden Kopien mit *zfs destroy* gelöscht - die ältesten zuerst. Für das zu sichernde Dataset werden zudem alle bis auf den letzten Snapshot gelöscht.

```
root@Ubuntu2510:/home/jakob/zfs-backup# python3 backup.py 4 zpool_for-saving/save-me zpool_for-backups/backups

--------------------
Starting Backup at: 2026-03-21T13:15:36
Source Folder: zpool_for-saving/save-me
Destination Folder: zpool_for-backups/backups
Deleting: zpool_for-backups/backups@2026-03-21T13:15:06
Deleting: zpool_for-backups/backups@2026-03-21T13:15:07
Deleting: zpool_for-saving/save-me@2026-03-21T13:15:12
Finished Backup at: 2026-03-21T13:15:36
Time elapsed: 0:00:00.138936 Seconds
--------------------
Remaining Snapshots:
zpool_for-backups/backups@2026-03-21T13:15:09
zpool_for-backups/backups@2026-03-21T13:15:10
zpool_for-backups/backups@2026-03-21T13:15:12
zpool_for-backups/backups@2026-03-21T13:15:36
zpool_for-saving/save-me@2026-03-21T13:15:36
root@Ubuntu2510:/home/jakob/zfs-backup#
```

Aufgabe 2: Datei-Inkonsistenzen trotz ZFS Snapshots

ZFS Snapshots halten den Read-Only-Zustand eines Datasets zu einem bestimmten Zeitpunkt fest. Durch CoW sind Snapshots atomar und lassen sich fast in Echtzeit erstellen. Sie benötigen erst eigenen Speicherplatz, sobald der Inhalt des Dateisystems vom Zustand des Snapshots abweicht. Geänderte Blöcke werden in sogenannten Transaction Groups gruppiert und atomar auf die Platte geschrieben. Erst wenn eine TXG erfolgreich commited wurde, wird sie ausgeführten Snapshots berücksichtigt. Bis dahin werden noch die alten Blöcke referenziert. Durch dieses Vorgehen wird in jedem Fall Filesystem-Konsistenz erreicht, was aber nicht zwangsläufig bedeutet, dass auch Applikationen oder Dateien logisch konsistent sind.

Um diese Inkonsistenz zu zeigen, wurden in der Funktion *write_file()* des durchgeführten Experiments (inconsistentSnapshot.py) 200x 1 MB Blöcke an Random-Bytes in eine .bin-Datei geschrieben. Hierbei werden mehrstufige Writes bzw. Transaktionen simuliert, indem jeder Random-Block zusätzlich von zwei weiteren Einträgen umschlossen wird, die den Start und das Ende des aktuellen Blocks mit seiner entsprechenden Nummer

festhalten. Logisch besteht ein ganzer Write also aus einem Header, dem Block und einem Footer. In einer weiteren Funktion *take_snapshot()* wird nach einem Sleep zwischen 1 und 5 Sekunden ein Snapshot ausgeführt. Beide Funktionen werden als separate Threads gestartet, sodass der Snapshot auslöst, während die Datei noch geöffnet bzw. geschrieben wird. Über eine Funktion *get_blocks()* werden sowohl vor als auch nach dem Rollback alle Block-STARTs und Block-ENDs mit dem aufgenommenen Snapshot gegrept und gezählt. Bei logisch konsistenten Writes sollte die Anzahl an STARTs und ENDs immer gleich sein.

```
root@Ubuntu2510:/home/jakob/zfs-backup/task2# python3 inconsistency.py
--------------------
Starting Execution at: 2026-03-21T15:22:16
Finished Snapshot!
Finished Execution at: 2026-03-21T15:22:23
Time elapsed: 0:00:07.181613 Seconds
Before rollback with zpool_for-saving/save-me@inconsistency
--------------------
Number of Start Blocks: 200
Number of End Blocks: 200
--------------------
Rolling back with taken snapshot!
After rollback with zpool_for-saving/save-me@inconsistency:
--------------------
Number of Start Blocks: 69
Number of End Blocks: 68
--------------------
```

Wie man im Screenshot sehen kann, ist die Datei zwar vielleicht Filesystem-konsistent, da keine der einzelnen Writes nur teilweise auf die Platte geschrieben worden ist, allerdings ist die logische Verknüpfung des mehrstufigen Writes gestört, da in Block 69 kein Footer vorhanden ist. Selbiges Prinzip lässt sich auch auf komplexere Transaktionen, wie z.B. bei Datenbanken anwenden.

```
root@Ubuntu2510:/home/jakob/zfs-backup/task2# python3 consistentSnapshot.py
--------------------
Starting Execution at: 2026-03-21T20:50:56
Finished Snapshot!
Finished Execution at: 2026-03-21T20:51:08
Time elapsed: 0:00:11.843204 Seconds
Before rollback with zpool_for-saving/save-me@consistency
--------------------
Number of Start Blocks: 200
Number of End Blocks: 200
--------------------
Rolling back with taken snapshot!
After rollback with zpool_for-saving/save-me@consistency:
--------------------
Number of Start Blocks: 56
Number of End Blocks: 56
--------------------
```

```
root@Ubuntu2510:/home/jakob/zfs-backup/task2# python3 fullyConsistentFile.py
--------------------
Starting Execution at: 2026-03-21T20:51:21
Finished Snapshot!
Finished Execution at: 2026-03-21T20:51:31
Time elapsed: 0:00:09.974555 Seconds
Before rollback with zpool_for-saving/save-me@fullConsistency
--------------------
Number of Start Blocks: 200
Number of End Blocks: 200
--------------------
Rolling back with taken snapshot!
After rollback with zpool_for-saving/save-me@fullConsistency:
--------------------
Number of Start Blocks: 200
Number of End Blocks: 200
--------------------
```

Um diesen Nachteil zu beheben, kann man unter anderem Gebrauch von Synchronisationstechniken wie z.B. Locks oder Lock-Files machen. Das Lock wird dabei zu Beginn einer Transaktion gesetzt und erst nach einer erfolgreichen Transaktion bestehend aus den drei einzelnen Writes für Header, Block und Footer wieder freigegeben. Alternativ könnte man auch zu Beginn jeder Iteration ein Lockfile erstellen und am Ende wieder entfernen. Bei Applikationen könnte man den Nachteil beheben, indem man die Anwendung kurz für den Snapshot pausiert und bei Datenbanken, indem man ein Readlock setzt und noch offene Transaktionen auf die Platte schreibt, bevor man den Snapshot ausführt. In den beiden obigen Screenshots wurde der Code von inconsistentSnapshot.py lediglich um ein Lock für jede einzelne Schreibiteration mit

kurzer Pause zwischen den Writes (links, consistentSnapshot.py) und einem Lock um den gesamten Schreibprozess (rechts, fullyConsistentFile.py) erweitert.

Aufgabe 3: Datensicherheit dank raidz und raidz2

```
  pool: zpool_raidz
 state: ONLINE
  scan: resilvered 100M in 00:00:01 with 0 errors on Sat Mar 21 21:17:08 2026
config:

        NAME         STATE     READ WRITE CKSUM
        zpool_raidz  ONLINE       0     0     0
          raidz1-0   ONLINE       0     0     0
            sdb      ONLINE       0     0     0
            sdc      ONLINE       0     0     0
            sdd      ONLINE       0     0     0

errors: No known data errors
```

```
root@Ubuntu25:/home/jakob/zfs-raidz# python3 raidz.py
--------------------
Running crash while Write simulation...
--------------------
Starting at: 2026-03-21T21:16:44
Start Writing...
Disk /dev/sdd crashed!
Finish Writing...
Number of Start Blocks: 200
Number of End Blocks: 200
Ending at: 2026-03-21T21:16:47
Elapsed time: 0:00:03.752832 seconds
--------------------
Replacing offline disk /dev/sdd with unused disk /dev/sdg
Running crash while Read simulation...
--------------------
Starting at: 2026-03-21T21:16:56
Start Reading...
Disk /dev/sdg crashed!
Finish Reading...
Number of Start Blocks: 200
Number of End Blocks: 200
Ending at: 2026-03-21T21:17:04
Elapsed time: 0:00:08.364649 seconds
--------------------
Replacing offline disk /dev/sdg with unused disk /dev/sdd
--------------------
Running no crash while Write simulation...
--------------------
Starting at: 2026-03-21T21:17:13
Start Writing...
Finish Writing...
Number of Start Blocks: 200
Number of End Blocks: 200
Ending at: 2026-03-21T21:17:16
Elapsed time: 0:00:03.739186 seconds
--------------------
Running no crash while Read simulation...
--------------------
Starting at: 2026-03-21T21:17:16
Start Reading...
Finish Reading...
Number of Start Blocks: 200
Number of End Blocks: 200
Ending at: 2026-03-21T21:17:24
Elapsed time: 0:00:07.636824 seconds
--------------------
```

Um raidz1 zu implementieren, benötigt man mindestens drei freie Block-Devices. Über den Befehl *zpool create mypool raidz1 /dev/sdb /dev/sdc /dev/sdd* lässt sich bspw. ein neuer Pool erstellen, der als vdev ein raidz1-Verbund aus drei Devices umfasst.

Um nachzuweisen, dass auch hier der Ausfall einer Platte keinen unmittelbaren Einfluss auf das Dateisystem oder laufende Lese- und Schreibvorgänge hat, werden analog zu Aufgabe 2 zuerst 200 MB an Random-Bytes in eine .bin-Datei geschrieben und im Anschluss wieder gelesen. Dieser Vorgang wird sowohl ohne als auch mit zwischenzeitigem Crash einer der Platten durchgeführt, wodurch jeweils zwei Messungen für Lese- und Schreibvorgänge entstehen, die miteinander verglichen werden können. Gestartet wird das Skript unter Angabe der „ausfallenden“ Platte, der Platte für das Resilvering und dem betrachteten Dataset: *python3 raidz.py /dev/sdd /dev/sdg zpool_raidz/raid*. Ausfall bedeutet im Falle der Implementierung die Ausführung des Befehls *zpool offline* und Resilvering bezeichnet das anschließende Ersetzen der Platte mit *zpool replace*. Als erstes werden Datenblöcke in eine Datei geschrieben, wobei nach einem kurzen Sleep die angegebene Platte ausfällt. Nach dem Resilvering der ersten Platte wird die zuvor geschriebene Datei gelesen, wobei währenddessen erneut eine Platte ausfällt. Nach dem auch diese wieder ausgetauscht wurde, folgt der gleiche Ablauf nur ohne zwischenzeitliche Ausfälle. Für alle vier Vorgänge wird die Zeit gemessen sowie die Anzahl der Start- und End-Blöcke gezählt. Hätte der Ausfall einer Platte eine Auswirkung auf das Dateisystem oder die Schreib-/Lesevorgänge müssten die Ergebnisse stark voneinander abweichen. Wie man aber im Screenshot erkennen kann, führt der Ausfall einer Platte zu keinen Filesystem-Inkonsistenzen und nur zu

minimal höheren Latenzen, die vermutlich auf das Ausführen des „Crash“-Befehls *zpool offline* zurückgeführt werden können. Da die Parität bei raidz1 dynamisch verteilt ist, macht es auch keinen Unterschied, welche der drei Platten aus dem Verbund ausfällt, solange es nicht mehr als eine ist. Mehr als eine ausfallende Platte gleichzeitig kann die Redundanz nicht auffangen.

Aufgabe 4: Zugriffsperformanz

Um die Zugriffsperformanz für ZFS und ext4 vergleichen zu können, wurde Gebrauch von verschiedenen Benchmarks des Flexible I/O Testers (fio) gemacht. Installiert wird dieses Tool mit *sudo apt install fio*. Für alle Benchmarks wurde ein Timout von 30 Sekunden sowie eine Block Size von einem Megabyte und einer zu transferierenden Datenmenge von zwei Gigabytes gewählt. Als I/O Patterns wurden sequentielle Reads, sequentielle Writes, randomisierte Reads, randomisierte Writes, gemischte sequentielle Reads und Writes sowie gemischte randomisierte Reads und Writes ausgeführt. Letztere wurden zusätzlich noch mit einer buffer_compress percentage von 60%, also teilweise komprimierbaren Daten, wiederholt. Alle Befehle wurden in einer .sh-Datei gebündelt und sowohl für ein ZFS-Dataset als auch ein mit mkfs.ext4 erstelltes und anschließend gemountetes ext4-Dateisystem ausgeführt. Alle Ergebnisse wurden als Textdatei gespeichert und liegen im GitHub-Repo innerhalb des Benchmark-Folders.

Seqwrite: ZFS minimal schneller mit BW: 402 MiB/s vs. 397 MiB/s

Seqread: ext4 leicht schneller mit BW: 430 MiB/s vs. 418 MiB/s

Randwrite: ext4 leicht schneller mit BW: 454 MiB/s vs. 444 MiB/s

Randread: ext4 deutlich schneller mit BW: 344 MiB/s vs. 251 MiB/s

Readwrite: ZFS outperformt bandweitentechnisch deutlich sowohl beim Read mit 818 MiB/s vs. 335 MiB/s als auch beim Write mit 867 MiB/s vs. 355 MiB/s

Randrw: kein Unterschied bandweitentechnisch sowohl beim Read mit 166 MiB/s als auch beim Write mit 176 MiB/s

Randrw_compress60: ZFS leicht bandweitentechnisch leicht besser beim Read mit 191 MiB/s vs. 162 MiB/s und beim Write mit 202 MiB/s vs. 171 MiB/s

Readwrite_compress60: ZFS bandweitentechnisch deutlich besser beim Read mit 750 MiB/s vs. 336 MiB/s und beim Write mit 795 MiB/s vs. 356 MiB/s

Die Ergebnisse scheinen insgesamt eher gemischt zu sein. Oft gibt es keine großen Unterschiede und nur leichte Tendenzen, allerdings scheint ZFS gerade bei gemischten sequentiellen Reads und Writes eine deutliche Outperformance gegenüber ext4 zu erreichen. Auch bei den komprimierbaren Daten scheint ZFS besser zu performen, was vermutlich an den standardmäßig verwendeten Komprimierungsalgorithmen wie LZ4 liegt.